

Reviewer Pack — Minimal Tropical Motivic Rank Invariance under Symplectic Re...

Entry 7eeb38edb185 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 02:00:09 UTC

Minimal Tropical Motivic Rank Invariance under Symplectic Reflection for Resolution Proof Width

Entry ID: 7eeb38edb185 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 02:00:09 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every d -regular graph G and its associated Tseitin formula φ_G , the minimal tropical motivic rank ($\text{mtr}(\varphi_G)$) is invariant under symplectic reflections of G , such that for any symplectically equivalent graph G' , $\text{mtr}(\varphi_G) = \Theta(\text{mtr}(\varphi_{G'}))$, where symplectic reflection is an operation that preserves the resolution proof width $w(\varphi_G)$.

Rationale (proposer's reasoning):

Tropical geometry offers a rich algebraic structure that could potentially provide insights into the complexity of resolution proofs. If minimal tropical motivic rank remains invariant under symplectic reflections, it may suggest a deep connection between geometric properties and computational complexity, providing a new tool for understanding the invariants that determine proof width.

Taxonomy category: TROPICAL_GEOMETRY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0666bf2bb579962c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all d -regular graphs G with $n \leq 40$ and their symplectically equivalent graph G' , $\text{mtr}(\varphi_G) = \Theta(\text{mtr}(\varphi_{G'}))$ holds true with a constant factor $k \leq 2$ across at least 30 random seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropical geometry" AND "resolution proof complexity" AND "minimal tropical motivic rank" - "symplectic reflection" AND "resolution proof width" AND "Tseitin formula" - "invariant under symplectic reflections" AND "tropical motivic rank invariance" AND "d-regular graph"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_graph(n, d):
        if (n * d) % 2 != 0:
            raise ValueError("Graph size must be a multiple of the degree")
        graph = {i: [] for i in range(n)}
        edges_added = set()
        for _ in range(d * n // 2):
            while True:
                u, v = random.sample(range(n), 2)
                if u == v or (u, v) in edges_added or (v, u) in edges_added:
                    continue
                graph[u].append(v)
                graph[v].append(u)
                edges_added.add((u, v))
            break
        return graph

    def is_symplectically_reflective(graph):
        n = len(graph)
        for i in range(n):
            for j in range(i + 1, n):
                if not (j in graph[i] and i in graph[j]):
                    return False
        return True

    def tseitin_formula(graph):
        n = len(graph)
```

```

literals = {i: f"x{i}" for i in range(n)}
clauses = []
for i in range(n):
    clause = [literals[i]]
    for j in graph[i]:
        clause.append(f"~{literals[j]}")
    clauses.append(clause)
return clauses

def mtr(formula):
    n = len(formula)
    if n == 0:
        return 0
    max_clause_length = max(len(clause) for clause in formula)
    return max_clause_length

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    graph = generate_d_regular_graph(n, 3)
    if not is_symplectically_reflective(graph):
        return {
            "metric_name": "mtr",
            "metric_value": -1,
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "Graph is not symplectically reflective"
        }
    phi_G = tseitin_formula(graph)
    mtr_phi_G = mtr(phi_G)

    for _ in range(5):
        graph_prime = generate_d_regular_graph(n, 3)
        if not is_symplectically_reflective(graph_prime):
            return {
                "metric_name": "mtr",
                "metric_value": -1,
                "instances_tested": 1,
                "n_max": n,
                "conjecture_holds": False,
                "counterexample": "Graph is not symplectically reflective"
            }
        phi_G_prime = tseitin_formula(graph_prime)
        mtr_phi_G_prime = mtr(phi_G_prime)

    if abs(mtr_phi_G - mtr_phi_G_prime) / max(1, mtr_phi_G, mtr_phi_G_prime) > 2:
        return {
            "metric_name": "mtr",
            "metric_value": -1,
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": f"mtr( $\phi_G$ ) = {mtr_phi_G}, mtr( $\phi_{G\_prime}$ ) = {mtr_phi_G_prime}"
        }

```

```

        results.append({
            "metric_name": "mtr",
            "metric_value": mtr_phi_G,
            "instances_tested": 6,
            "n_max": n,
            "conjecture_holds": True,
            "counterexample": ""
        })

    return {
        "metric_name": "mtr",
        "metric_value": sum(result["metric_value"] for result in results) / len(results),
        "instances_tested": sum(result["instances_tested"] for result in results),
        "n_max": max(result["n_max"] for result in results),
        "conjecture_holds": all(result["conjecture_holds"] for result in results),
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='Graph is not symplectically reflective' first_fa
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b7a67adb.py", line 129, in <module>

```

    result = run_trial(seed)
    ~~~~~
```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b7a67adb.py", line 67, in run_trial

```

    graph = generate_d_regular_graph(n, 3)
    ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b7a67adb.py", line 23, in generate_d_re
    raise ValueError("Graph size must be a multiple of the degree")
ValueError: Graph size must be a multiple of the degree
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Investigate and fix the error in the test code that caused it to crash. Once the issue is resolved, rerun the test with a valid graph pair to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16479
2	preregistration	ollama_remote	glm4:latest	0	0	10207
3	novelty	ollama_remote	glm4:latest	0	0	8564
4	novelty	ollama_remote	glm4:latest	0	0	9012
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14062
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12370
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16339
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14280
9	judge	ollama_remote	glm4:latest	0	0	16374

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 117686 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/7eeb38edb185.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/7eeb38edb185.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/7eeb38edb185.tar.gz (if generated)